Indian Institute of Information Technology Bhagalpur

Department of Electronics and Communication Engineering

REPORT

on

Design and Simulation of a Finite Impulse Response (FIR) Digital Filter using Verilog In ModelSim

Submitted by:

Nitish Kumar Thakur

Roll No.: 250202011

Semester: 2nd

Course: SoC Design using Verilog

Date of Submission: ..../05/2026

Academic Year: 2025–2026

Introduction

Digital filters are widely used in signal processing applications such as noise reduction, audio processing, and communication systems. FIR filters are preferred due to their inherent stability and simple structure.

In this project, a FIR filter is designed using Verilog HDL and verified through simulation. The filter uses a fixed number of coefficients and processes discrete-time input signals.

Objective

- To design a FIR digital filter using Verilog
- To implement a 4-tap filter structure
- To verify the design using simulation
- To analyze the output behavior

Theory

A FIR filter computes output using present and past input samples. The general equation is:

FIR Filter Block Diagram

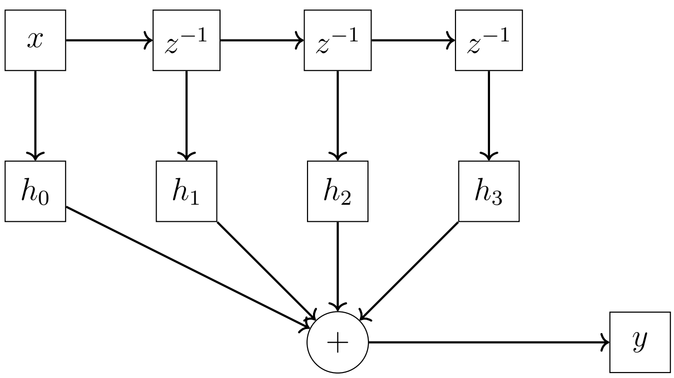


Figure 1: Block diagram of 4-tap FIR filter

$$y[n] = h_0x[n] + h_1x[n-1] + h_2x[n-2] + h_3x[n-3] \tag{1}$$

where:

- $x[n]$ = current input
- h_0, h_1, h_2, h_3 = filter coefficients

In this project, coefficients are chosen as: [$\mathrm{h}_0 = 1, \quad h_1 = 2, \quad h_2 = 3, \quad h_3 = 4$]

Design Methodology

The FIR filter is implemented using:

- Shift registers to store past inputs
- Multipliers for coefficient multiplication
- Adders to compute the final output

The design is synchronous and operates on the rising edge of the clock.

Verilog Implementation

The FIR filter is implemented using Verilog HDL. The design includes:

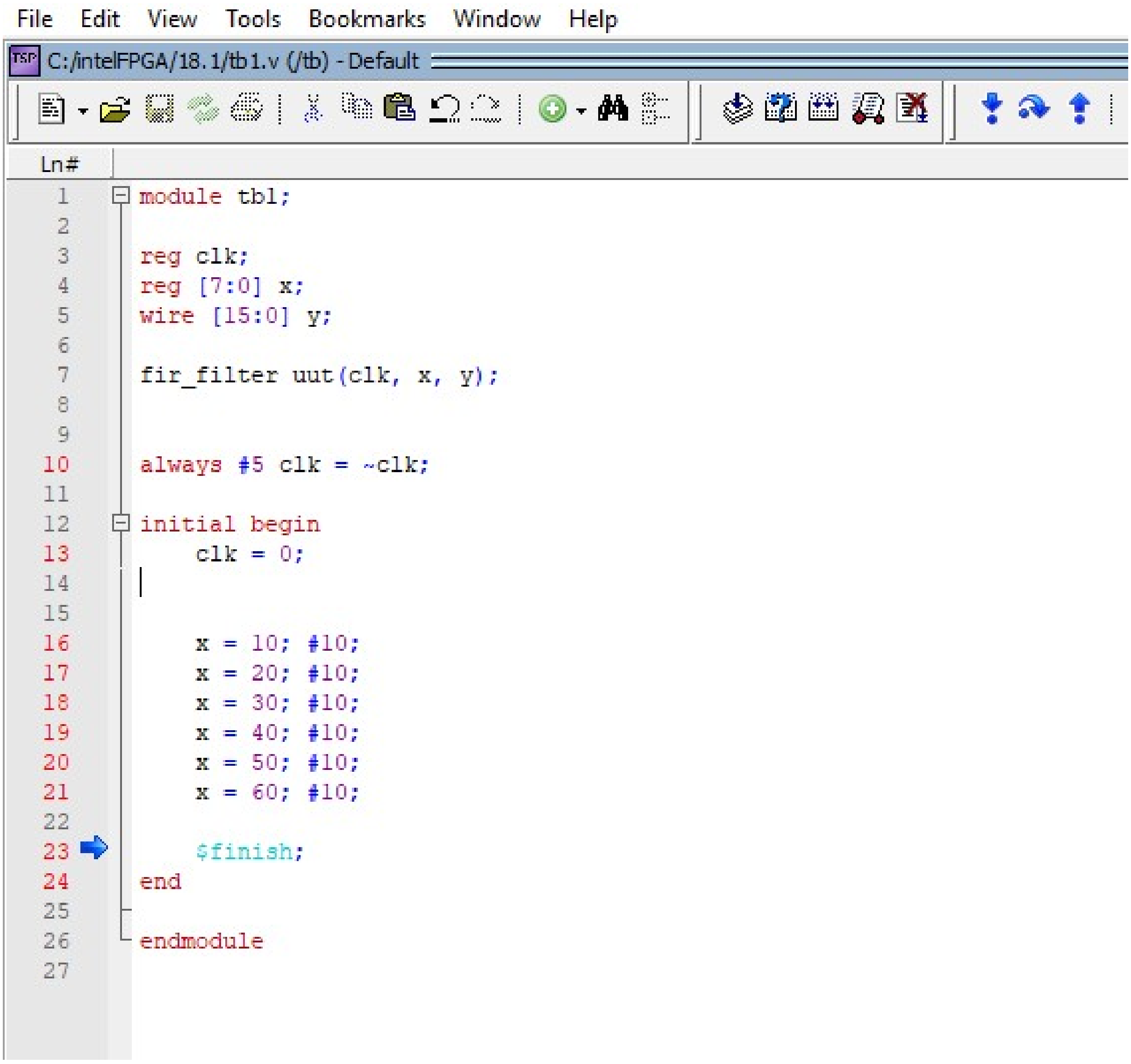
- Input signal
- Delay elements (registers)
- Output calculation logic

FIR Filter Code

```
module fir_filter(x,y,clk);
    input clk,
    input [7:0] x,
    output reg [15:0] y


reg [7:0] x1, x2, x3;


parameter h0 = 1;
parameter h1 = 2;
parameter h2 = 3;
parameter h3 = 4;

always @(posedge clk) begin

    x3 <= x2;
    x2 <= x1;
    x1 <= x;

    y <= (h0 * x) + (h1 * x1) + (h2 * x2) + (h3 * x3);
end

endmodule
```

Figure 2: FIR Filter Code

FIR Filter Testbench




```verilog
module tb1;

reg clk;
reg [7:0] x;
wire [15:0] y;

fir_filter uut(clk, x, y);


always #5 clk = ~clk;

initial begin
    clk = 0;


    x = 10; #10;
    x = 20; #10;
    x = 30; #10;
    x = 40; #10;
    x = 50; #10;
    x = 60; #10;

    $finish;
end

endmodule
```

Figure 3: FIR Filter Testbench

Simulation Setup

The design is simulated using ModelSim. A testbench is created to apply input sequences and observe the output waveform.

Input sequence used: [10, 20, 30, 40, 50, 60]

- The simulation was performed for 100 ns using ModelSim.

Simulation Results

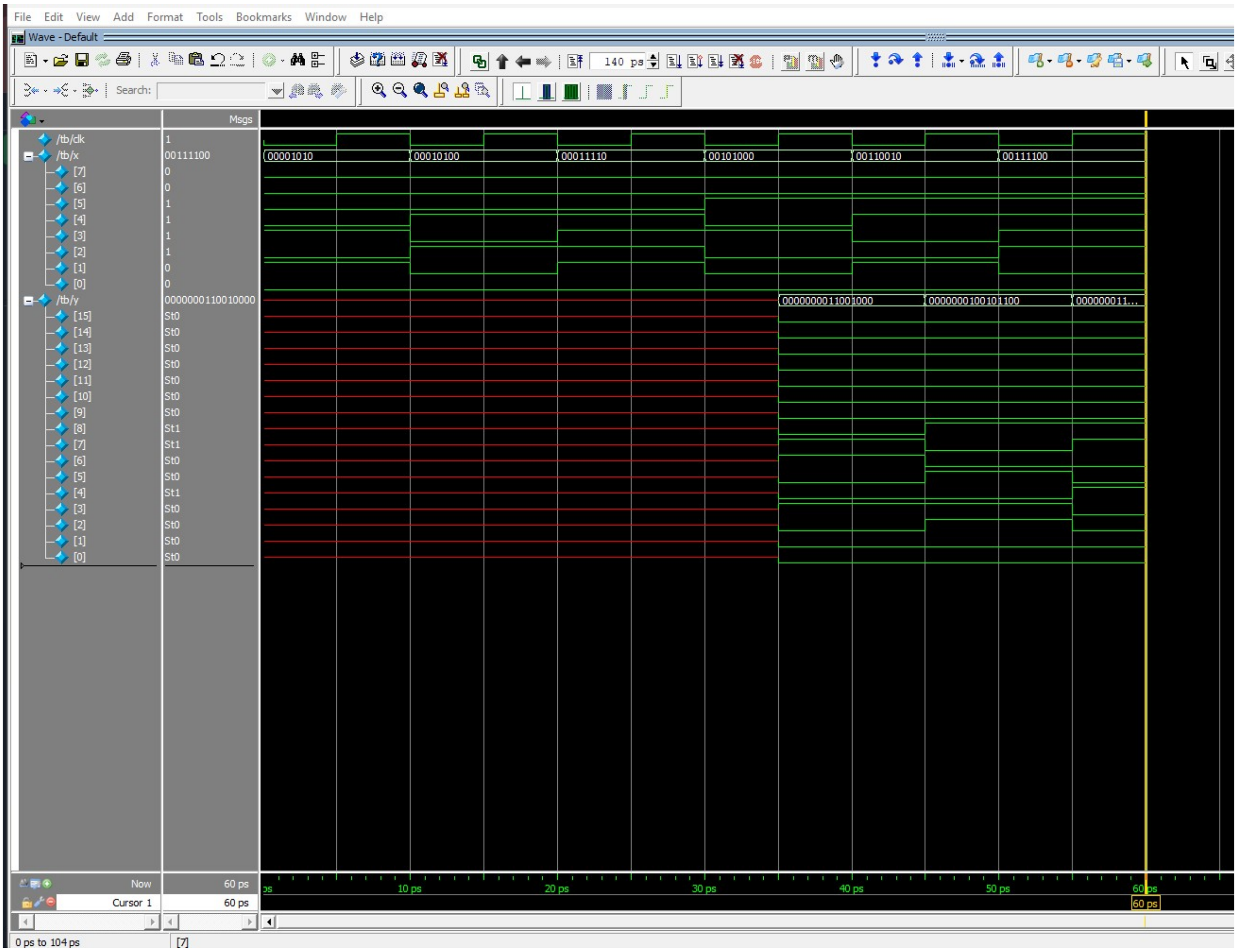


Figure 4: FIR Filter Waveform(ModelSim)

Results and Discussion

Initially output is zero due to empty delay registers. After a few clock cycles, the FIR filter produces correct output using current and past inputs.

The simulation results show that:

- Output depends on both current and previous inputs
- Output increases gradually due to accumulation
- Filter behaves as expected for FIR system

The waveform confirms correct implementation of the FIR filter.

Advantages

- Stable system (no feedback)
- Simple implementation
- Suitable for real-time signal processing

Applications

- Audio signal processing
- Image filtering
- Communication systems
- Noise reduction

Conclusion

The FIR filter has been successfully designed and simulated using Verilog HDL. The output behavior confirms that the filter correctly processes current and past input samples. The FIR filter design confirms that digital filtering can be efficiently implemented using hardware description languages like Verilog.

Future Scope

- Implement higher-order FIR filters
- Use programmable coefficients
- Implement on FPGA hardware